Problem Set 2

Name: Nicolas Iskandar

Problem 2-1.

(a) Recursion Tree. Build a tree where the root does $O(n)$ work and branches into 4 children, each on a subproblem of size $n/2$. At depth i there are 4^i nodes, each of size $n/2^i$ and doing $O(n/2^i)$ work, so the total work at depth i is

$$4^i \cdot O\left(\frac{n}{2^i}\right) = O(2^i n).$$

The recursion bottoms out at depth $\log_2 n$, so summing a geometric series that is dominated by its last term gives

$$\sum_{i=0}^{\log_2 n} O(2^i n) = O(n \cdot 2^{\log_2 n}) = O(n^2).$$

The number of leaves is $4^{\log_2 n} = n^2$, each contributing $\Theta(1)$ work, so the leaves alone already force $\Omega(n^2)$ work. Combining the two bounds, $T(n) = \Theta(n^2)$.

Master Theorem. With $a = 4$, $b = 2$, $f(n) = O(n)$, we compare $f(n)$ against $n^{\log_b a} = n^2$. Since $n = O(n^{2-\varepsilon})$ for $\varepsilon = 1$, we are in Case 1, giving

$$\boxed{T(n) = \Theta(n^2).}$$

(b) Recursion Tree. With $a = 3$, $b = \sqrt{2}$, $f(n) = O(n^4)$: at depth i there are 3^i nodes, each of size $n/(\sqrt{2})^i$ and costing $O\big((n/(\sqrt{2})^i)^4\big) = O(n^4/4^i)$. The total work at depth i is

$$3^i \cdot O\left(\frac{n^4}{4^i}\right) = O\left(n^4\left(\frac{3}{4}\right)^i\right).$$

Because $3/4 < 1$, this is a decreasing geometric series in i, so its sum over all $\log_{\sqrt{2}} n = 2\log_2 n$ levels is dominated by the root term:

$$\sum_{i=0}^{2\log_2 n} O\left(n^4\left(\tfrac{3}{4}\right)^i\right) = O(n^4).$$

The root alone costs $\Theta(n^4)$, so this matches the upper bound and $T(n) = O(n^4)$. For the lower bound, simply note the root by itself costs $\Omega(n^4)$ (dropping all recursive calls only decreases the total), so $T(n) = \Omega(n^4)$ as well. Hence $T(n) = \Theta(n^4)$.

Master Theorem. With $a = 3$, $b = \sqrt{2}$, we get $n^{\log_b a} = n^{\log_{\sqrt{2}} 3} = n^{2\log_2 3} \approx n^{3.17}$. Since $f(n) = \Theta(n^4)$ and $4 > 2\log_2 3$, we check the regularity condition for Case 3:

$$af(n/b) = 3\left(\frac{n}{\sqrt{2}}\right)^4 = 3 \cdot \frac{n^4}{4} = \tfrac{3}{4}n^4 \le c\,n^4 \quad \text{for } c = \tfrac{3}{4} < 1.$$

The condition holds, so Case 3 applies and

$$\boxed{T(n) = \Theta(n^4).}$$

(c) Recursion Tree. At depth i there are 2^i nodes of size $n/2^i$, each costing $5 \cdot \frac{n}{2^i} \log \frac{n}{2^i}$. Writing $\log(n/2^i) = \log n - i$, the total work at depth i is

$$2^i \cdot 5\frac{n}{2^i}(\log n - i) = 5n(\log n - i).$$

Reindexing with $j = \log n - i$ (so j runs from $\log n$ down to 0 as i runs from 0 to $\log n$), the total cost over the whole tree is

$$\sum_{j=0}^{\log n} 5nj = 5n \cdot \frac{(\log n)(\log n + 1)}{2} = \Theta(n \log^2 n).$$

Master Theorem (Case 2, extended form). Here $a = b = 2$ so $n^{\log_b a} = n$, and $f(n) = 5n \log n = \Theta(n^{\log_b a} \log^k n)$ with $k = 1$. The extended Case 2 gives $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$, so

$$\boxed{T(n) = \Theta(n \log^2 n).}$$

(d) Claim: $T(n) = \Theta(n^2)$.

Intuition. Unrolling once gives $T(n) = \Theta(n) + T(n-2)$, and repeating this $\Theta(n)$ times peels off terms $\Theta(n), \Theta(n-2), \Theta(n-4), \ldots, \Theta(1)$, an arithmetic-like sum of $\Theta(n)$ terms averaging $\Theta(n)$ each, so we expect $\Theta(n^2)$ total. Substitution turns this guess into a proof by induction: we assume the guessed bound holds for the smaller subproblem $T(n-2)$, substitute it into the recurrence, and check that the algebra closes for some valid choice of constant.

Let $d > 0$ be the constant hidden inside $\Theta(n)$, so concretely

$$T(n) = T(n-2) + dn.$$

Upper bound: $T(n) = O(n^2)$. **Guess:** $T(n) \le cn^2$ for some constant $c > 0$ and all $n \ge n_0$.

Inductive hypothesis: assume the guess already holds for the smaller input $n-2$, i.e. $T(n-2) \leq c(n-2)^2$.

Substitute this into the recurrence:

$$T(n) = T(n-2) + dn \;\leq\; c(n-2)^2 + dn.$$

Expand $c(n-2)^2 = cn^2 - 4cn + 4c$, so

$$T(n) \;\leq\; cn^2 - 4cn + 4c + dn \;=\; cn^2 + (d-4c)n + 4c.$$

We want to conclude $T(n) \leq cn^2$, which holds exactly when the leftover terms satisfy

$$(d-4c)n + 4c \;\leq\; 0.$$

Choose c**:** pick $c = d$, so $d - 4c = -3d < 0$. The leftover expression becomes $-3dn + 4d$, a downward-sloping line in n that is ≤ 0 once

$$n \;\geq\; \tfrac{4}{3}.$$

So for all $n \geq n_0 = 2$ (checking the base case $T(1) = \Theta(1)$ separately, absorbed by choosing c large enough), the leftover term is non-positive, giving $T(n) \leq cn^2$ as desired. By induction this holds for all $n \geq n_0$, so $T(n) = O(n^2)$.

Lower bound: $T(n) = \Omega(n^2)$. **Guess:** $T(n) \geq cn^2$ for some (possibly different, smaller) constant $c > 0$.

Inductive hypothesis: assume $T(n-2) \geq c(n-2)^2$.

Substitute and expand exactly as before:

$$T(n) = T(n-2) + dn \;\geq\; c(n-2)^2 + dn \;=\; cn^2 + (d-4c)n + 4c.$$

Now we want the leftover terms $(d-4c)n + 4c$ to be ≥ 0, so that $T(n) \geq cn^2$ follows directly.

Choose c**:** pick $c = d/8$, so $d - 4c = d - d/2 = d/2 > 0$. Then the leftover term $\frac{d}{2}n + 4c$ is positive for every $n \geq 0$ automatically, no need to wait for n to be large. Hence $T(n) \geq cn^2$ for all $n \geq 1$, so $T(n) = \Omega(n^2)$.

Conclusion. Combining $T(n) = O(n^2)$ and $T(n) = \Omega(n^2)$,

$$\boxed{T(n) = \Theta(n^2).}$$

Problem 2-2.

(a) Algorithm: Selection sort.

The cost of each sort is dominated by how it uses `get_at` (cheap, $\Theta(1)$) versus `set_at` (expensive, $\Theta(n \log n)$), so the right choice is whichever algorithm minimizes total *writes*, not comparisons.

- *Selection sort* performs $\Theta(n^2)$ reads to find each minimum but only n writes total (one placement per pass): cost $\Theta(n^2) + \Theta(n) \cdot \Theta(n \log n) = \Theta(n^2 \log n)$.
- *Insertion sort* can perform up to $\Theta(n^2)$ writes in the worst case (shifting elements one at a time), giving $\Theta(n^2) \cdot \Theta(n \log n) = \Theta(n^3 \log n)$.
- *Merge sort* is not in-place (it needs auxiliary storage to merge), so it is disqualified by the problem's requirement.

Since writes are the bottleneck resource here, **selection sort** wins with $\Theta(n^2 \log n)$.

(b) Algorithm: Merge sort.

Here reads/writes are free but comparisons cost $\Theta(\log n)$ each, so the deciding factor is the number of comparisons each algorithm performs.

- Selection sort and insertion sort each make $\Theta(n^2)$ comparisons in the worst case, for a total cost of $\Theta(n^2 \log n)$.
- Merge sort makes only $\Theta(n \log n)$ comparisons (each merge step compares once per output element), for a total cost of $\Theta(n \log n) \cdot \Theta(\log n) = \Theta(n \log^2 n)$.

Since $n \log^2 n = o(n^2 \log n)$, **merge sort** is asymptotically best, at $\Theta(n \log^2 n)$.

(c) Algorithm: Insertion sort.

The key resource to minimize here is total work on a nearly-sorted input, and insertion sort is the only one of the three whose running time is sensitive to how far the input already is from sorted.

Each of the $\log \log n$ adjacent swaps can create at most one new inversion, so the array has at most $\log \log n$ inversions total after the disruption (it started with zero, being sorted). Insertion sort's running time is $\Theta(n+I)$ where I is the number of inversions in the input, since the inner `while` loop only performs work proportional to the number of inversions it resolves, plus $\Theta(1)$ per outer iteration. Thus insertion sort runs in

$$\Theta(n + \log \log n) = \Theta(n).$$

Selection sort always costs $\Theta(n^2)$ and merge sort always costs $\Theta(n \log n)$ irrespective of how close to sorted the input already is, so both are worse here.

Therefore, **insertion sort** is best, at $\Theta(n)$.

Problem 2-3.

Algorithm (bidirectional exponential search).

Datum sits at kilometer k or kilometer $n - k$ for some unknown $k \geq 0$, and we want a strategy whose cost scales with k rather than with n, since the island's ends are most urgent.

Phase 1 (bracketing). Probe alternately from the north and south ends at exponentially growing offsets. Concretely, for $j = 0, 1, 2, \ldots$:

- Teleport to kilometer 2^j (measuring from the north). Query the tracker.
- Teleport to kilometer $n - 2^j$ (measuring from the south). Query the tracker.

Stop as soon as one of these two probes "overshoots" Datum, i.e., the tracker reports Datum is now on the side of the probe closer to the relevant end. At that point we know Datum lies between the previous probe on that side (at offset 2^{j-1}, or 0 if $j = 0$) and the current one (at offset 2^j).

Since Datum is k kilometers from the nearer end, the overshoot happens at $j^* = \lceil \log_2 k \rceil$, and we have made $O(j^*) = O(\log k)$ probes (both north and south probes together, since they interleave one-for-one).

Phase 2 (refinement). The bracket identified in Phase 1 has width at most $2^{j^*} = O(k)$. Run ordinary binary search inside this bracket, which takes $O(\log(2^{j^*})) = O(\log k)$ further probes to pin down Datum's exact kilometer.

Correctness. Phase 1 always terminates because probes march monotonically toward the center and Datum is at a fixed finite position, so eventually a probe passes Datum's position. The resulting bracket is guaranteed (by the tracker's directional guarantee) to contain Datum's true location, so binary search within it correctly isolates the exact kilometer.

Running time. Phase 1 costs $O(\log k)$ teleports, Phase 2 costs $O(\log k)$ teleports, for a total of $O(\log k)$, as required. ■

Problem 2-4.

Design goal. Every operation is simple except `ban`: when a viewer is banned, *all* of their previously sent messages must vanish from the chat, and doing this by scanning the whole message history would be far too slow. The whole design problem reduces to: *how do we delete one viewer's scattered messages without searching the entire chat?* The answer is to give each viewer a private index of pointers directly to their own messages, so a ban only ever touches that viewer's own messages, never anyone else's.

Data structures.

1. A **lookup structure** H (a balanced BST, e.g. an AVL tree, keyed on viewer ID, to guarantee worst-case $O(\log n)$ operations) mapping each viewer v to a record storing v's banned flag and the head of v's private list L_v.
2. A **global doubly-linked list** M containing every currently undeleted message, in chronological order (newest at one fixed end).
3. For each viewer v, a **private singly-linked list** L_v that stores, in order, *pointers directly into the nodes of* M corresponding to v's own messages, not copies of the messages, but pointers to their exact location in M.

The key idea is layer (3): because L_v points directly at v's nodes inside M, deleting all of v's messages never requires searching M, we already know exactly where each one lives.

Operations.

build(V): $O(n \log n)$. Insert all n viewer records into H one at a time; each insertion costs $O(\log n)$, for $O(n \log n)$ total. Initialize M and every L_v as empty in $O(1)$ each.

send(v, m): $O(\log n)$. Look up v's record in H, $O(\log n)$; if banned, do nothing. Otherwise, create a node for m, splice it onto the newest end of M in $O(1)$, and push a pointer to that same node onto the front of L_v in $O(1)$. Total: dominated by the lookup, $O(\log n)$.

recent(k): $O(k)$. Since M only ever contains currently undeleted messages, in chronological order, simply walk backward from the newest end of M, collecting message contents one at a time ($O(1)$ per step), stopping after k nodes (or when M is exhausted). Total: $O(k)$.

ban(v): $O(n_v + \log n)$. Look up v's record in H ($O(\log n)$) and mark it banned. Walk L_v: for each of the n_v pointers stored there, we already know exactly which node of M to remove, so splice it out of the doubly-linked list M in $O(1)$ using its stored neighbor pointers (no searching needed). Total: $O(n_v + \log n)$.

Correctness. `recent` only ever traverses nodes still linked into M, and `ban` removes exactly (and immediately) all of a banned viewer's nodes from M the moment they are banned, so a banned viewer's messages can never appear in a later `recent` call. All chronological ordering and per-viewer bookkeeping invariants are preserved by every operation, since `send` only ever appends to the newest end of M and the front of L_v.

Problem 2-5.

(a) Algorithm. We think of a booking schedule as a piecewise-constant "staircase" function $f(t)$ giving the number of rooms reserved at time t; a schedule $(k_1, s_1, t_1), \ldots, (k_m, s_m, t_m)$ is just f chopped into its maximal constant-value pieces (with $f = 0$ implicitly outside the listed intervals). Since B must satisfy $R = R_1 \cup R_2$, and every talk in R came from exactly one of R_1, R_2, the number of overlapping talks at any time t is additive:

$$f_B(t) = f_{B_1}(t) + f_{B_2}(t).$$

So it suffices to compute this sum, expressed as a booking schedule.

We compute it with a linear two-pointer sweep, analogous to merging two sorted lists. Maintain pointers i_1, i_2 into B_1, B_2 (initially 0) and a cursor x marking the rightmost time already emitted into the output (initially $-\infty$, or the minimum possible start time). Repeat until both pointers reach the end of their lists:

- If one list is exhausted, emit the current booking of the other list as-is (skipping any gap up to its start time via $x \leftarrow \max(x, s)$), and advance that pointer.
- Otherwise let (k_1, s_1, t_1) and (k_2, s_2, t_2) be the current bookings of B_1, B_2. Advance x past any joint gap: $x \leftarrow \max(x, \min(s_1, s_2))$. Then:
  - If $t_1 \leq s_2$ (or symmetrically $t_2 \leq s_1$), the two bookings don't overlap; emit (k_1, x, t_1) and advance i_1 (resp. emit (k_2, x, t_2), advance i_2).
  - Else if $x < s_2$ (resp. $x < s_1$), only B_1's (resp. B_2's) booking is active so far; emit (k_1, x, s_2) (resp. (k_2, x, s_1)) *without* advancing a pointer.
  - Otherwise both bookings are simultaneously active; emit $(k_1{+}k_2,\ x,\ \min(t_1, t_2))$, and advance whichever pointer(s) end exactly at $\min(t_1, t_2)$.

Finally, do a single linear pass merging any two adjacent emitted pieces that share the same room count and abut in time (this can occur because a booking may be emitted across several iterations); this restores the invariant that consecutive pieces of a valid schedule have distinct room counts.

Correctness. At every point in the sweep, x marks the boundary up to which $f_{B_1} + f_{B_2}$ has been fully and correctly emitted as a sequence of constant pieces: each case above identifies the next time at which either f_{B_1}, f_{B_2}, or their sum changes value, and emits the constant piece up to that point with the correct combined height k_1, k_2, or $k_1 + k_2$. Since every relevant breakpoint of B_1 and B_2 is eventually reached by one of i_1, i_2, the full sum $f_{B_1} + f_{B_2}$ is reconstructed, which is precisely f_B for B satisfying $R_1 \cup R_2$. The cleanup pass only merges pieces that are truly one contiguous constant-height run split across iterations, so it preserves f_B exactly while restoring the schedule invariant.

Running time. Each iteration of the sweep does $O(1)$ work and emits one piece to the output. An iteration advances at least one of i_1, i_2 *except* for the "only one side active" case, which does not advance a pointer but strictly increases x to a fixed boundary (s_1 or s_2); this case cannot repeat on consecutive iterations without an intervening pointer

advance, so it contributes only $O(1)$ extra iterations per pointer advance. Hence the sweep runs in $O(|B_1|+|B_2|) = O(n)$ time, and the final cleanup pass is a single linear scan, also $O(n)$. Total: $O(n)$.

(b) Algorithm. Apply divide and conquer, exactly as in merge sort, using part (a) as the combine step. Given R with $|R| = n$:

- If $n = 1$, say $R = \{(s, t)\}$, return the trivial schedule $B = ((1, s, t))$.
- Otherwise split R arbitrarily into two halves R_1, R_2 of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, recursively compute the satisfying schedules B_1 for R_1 and B_2 for R_2, and return $B = \textsc{Merge-Bookings}(B_1, B_2)$ from part (a).

Correctness. By induction on n: the base case is immediate. For the inductive step, B_1 and B_2 are, by the inductive hypothesis, the correct satisfying schedules for R_1 and R_2 respectively. Since $R = R_1 \cup R_2$ (as the split partitions R), part (a) guarantees that merging B_1 and B_2 yields the correct satisfying schedule for R.

Running time. Note $|B_i| \leq |R_i|$ for any request set (a schedule has at most one piece per request boundary), so the merge at a node handling n' requests costs $O(n')$ by part (a). This gives the familiar recurrence

$$T(n) = 2T(n/2) + O(n),$$

which by the Master Theorem (or a recursion-tree argument identical to merge sort's) solves to

$$T(n) = O(n \log n).$$